

High-Level Design (HLD) Fundamentals

Class Notes, Design Document Template & Homework

System Design Series — Session 1

Table of Contents

Table of Contents	2
1. What We Covered in This Class	3
2. Design Documents: Culture and Practice	3
3. Design Document Template	4
3.1 Problem Statement	4
3.2 Critical User Journeys (CUJs)	4
3.3 Timeline and Task Breakdown	4
3.4 High-Level Design (HLD)	5
3.5 Low-Level Design (LLD)	5
3.6 Test Plan and Bug Hunt	5
3.7 Release Plan	6
3.8 Success Criteria	6
3.9 Optional Sections (add as complexity demands)	6
4. System Design Fundamentals: How the Internet Works	7
5. Categories of Databases and Data Infrastructure	8
Homework Exercise 1: Go Deep on One Technology per Category	9
6. Cost Estimation in High-Level Design	10
7. Homework Exercise 2: Explore Cloud Providers	10
8. Homework Summary	11

1. What We Covered in This Class

This session introduced the fundamentals of High-Level Design (HLD) and the engineering culture around design documents at large technology companies. The class covered four broad areas:

- Design documents — what they are, who writes them, and a reusable template you can adopt for your own projects.
- System design fundamentals — starting with understanding how the internet works end to end when you hit a URL.
- Categories of databases and data infrastructure — and a homework exercise to go deep on one popular technology per category.
- Cost estimation and cloud awareness — why cost is a first-class consideration in HLD, plus a hands-on exercise with AWS, Azure, and GCP free tiers.

2. Design Documents: Culture and Practice

A design document is the primary artifact through which engineering decisions are proposed, reviewed, and recorded. At companies like Google and Meta, design documents are typically authored by staff-level or senior engineers who own the technical direction of a project. Writing the document is not a formality — it is how the author forces clarity of thought, surfaces trade-offs early, and gets alignment from the team and cross-functional stakeholders before any significant code is written.

A few practices worth internalizing from how large companies operate:

- Multiple templates exist within a company. There is no single universal format — templates vary with project complexity. A small feature may need a two-page 'mini design', while a new service or platform may need a full document with capacity planning and multi-quarter timelines.
- Each team adapts the template to its domain. A front-end team's design doc emphasizes user journeys, component structure, and accessibility; a back-end team's doc emphasizes APIs, data models, scaling, and reliability; an infra or ML team will have yet another shape.
- Design docs are reviewed, not just read. Senior engineers and peers comment, challenge assumptions, and approve. The document becomes a historical record of why decisions were made.

3. Design Document Template

Below is a reusable template. Copy it into a Google Doc (or your team's wiki) and fill in each section. Sections can be trimmed or expanded based on project complexity — the headings and guiding questions matter more than the exact length.

Document header (fill in at the top of every design doc)

Title • Author(s) • Reviewers / Approvers • Status (Draft / In Review / Approved) • Last Updated • Links (PRD, tickets, dashboards, related docs)

3.1 Problem Statement

State the problem in plain language. This section must answer, explicitly:

- What problem are we solving? Describe the current pain — for users, for the business, or for engineering.
- Why are we doing this project? Tie it to a user need, business goal, or engineering necessity (e.g., scaling limits, tech debt, compliance).
- Why now? What makes this urgent or timely — a launch deadline, growth trajectory, competitive pressure, an expiring system, new capability that unblocks it?
- What happens if we do nothing? The cost of inaction is often the strongest justification.
- Goals and non-goals. List what this project will deliver, and just as importantly, what is explicitly out of scope.

3.2 Critical User Journeys (CUJs)

Describe the key journeys a user takes through the product that this project affects. For each CUJ, write the step-by-step flow (entry point → actions → outcome), and note which steps change with this design. Good CUJs keep the design grounded in real user experience and later become the basis for end-to-end tests and success metrics. Include mocks, wireframes, or flow diagrams where helpful.

3.3 Timeline and Task Breakdown

Break the project into milestones and distribute tasks across the engineers working on it. Each task should have an owner, an estimate, and dependencies. A simple table works well:

Milestone	Task	Owner	Estimate	Dependencies	Target Date
M1: Foundation	Schema design & migration	Engineer A	1 week	—	—
M1: Foundation	API contract & stubs	Engineer B	1 week	—	—
M2: Build	Core service implementation	Engineer A	2 weeks	M1	—

Milestone	Task	Owner	Estimate	Dependencies	Target Date
M2: Build	Front-end integration	Engineer C	2 weeks	API stubs	—
M3: Hardening	Load testing & bug hunt	Whole team	1 week	M2	—
M4: Launch	Staged rollout	Engineer B	1 week	M3	—

3.4 High-Level Design (HLD)

The architectural view of the system. Cover:

- Architecture diagram — major components (clients, services, databases, queues, caches, third-party systems) and how they interact.
- Data flow — how a request travels through the system for the main CUIs.
- Technology choices — which database, queue, cache, framework, and why. Include alternatives considered and why they were rejected.
- Scale and capacity estimates — expected QPS, storage growth, read/write ratios.
- Cost estimation — rough infrastructure cost of the proposed design (see Section 6).
- Non-functional requirements — availability targets, latency budgets, security, privacy, compliance.

3.5 Low-Level Design (LLD)

The implementation-level detail for each component:

- API definitions — endpoints/RPCs, request and response schemas, error handling.
- Data models — tables/collections, indexes, key design.
- Class/module structure and key algorithms or state machines.
- Edge cases, failure modes, retries, idempotency.

3.6 Test Plan and Bug Hunt

Describe how the project will be verified at every level:

- Unit tests — what modules are covered, target coverage, key scenarios.
- Integration tests — which component boundaries and contracts are exercised.
- End-to-end tests — automated tests that walk the critical user journeys from Section 3.2.

Bug hunt plan: before release, the whole team comes together for a structured bug hunt. Document:

- Scope and setup — which environment/build to test, test accounts, seed data, feature flags to enable.
- Step-by-step test scenarios — the exact steps each participant should follow to exercise the project (happy paths, edge cases, error states, different devices/locales).

- Logistics — date/time, participants, how to file bugs (labels, priority rubric), and who triages.
- Exit criteria — e.g., no open P0/P1 bugs, all CUIs pass.

3.7 Release Plan

- Rollout strategy — staged rollout (e.g., 1% → 10% → 50% → 100%), feature flags, A/B experiment setup if applicable.
- Monitoring and alerting — dashboards and metrics to watch during rollout.
- Rollback plan — how to revert quickly and safely if something goes wrong.
- Communication — who needs to be informed (support, PM, partner teams) and when.

3.8 Success Criteria

Define, up front, how you will know the project succeeded:

- Product/business metrics — e.g., adoption, conversion, engagement targets.
- Technical metrics — latency (p50/p95/p99), error rates, availability, cost per request.
- Measurement plan — which dashboards/experiments will report these, and when the team will review results post-launch.

3.9 Optional Sections (add as complexity demands)

- Alternatives considered (with trade-off analysis)
- Security, privacy, and compliance review
- Open questions and risks
- Appendix / glossary / references

4. System Design Fundamentals: How the Internet Works

Before designing large systems, you must understand what actually happens when you type a URL into a browser and press Enter. Homework: trace this journey end to end and be able to explain every step. At a high level:

1. **URL parsing:** the browser breaks the URL into scheme (https), domain, path, and query parameters.
2. **DNS resolution:** the domain is translated to an IP address — checking browser/OS caches first, then the resolver, root, TLD, and authoritative name servers.
3. **TCP connection:** the client establishes a connection to the server IP (the three-way handshake: SYN, SYN-ACK, ACK).
4. **TLS handshake:** for HTTPS, client and server negotiate encryption and verify the server's certificate.
5. **HTTP request:** the browser sends a request (method, headers, cookies, body).
6. **Server-side processing:** the request may pass through a CDN, load balancer, and reverse proxy before reaching application servers, which talk to caches, databases, and other services to build the response.
7. **HTTP response:** the server returns a status code, headers, and body (HTML, JSON, etc.).
8. **Rendering:** the browser parses HTML, fetches CSS/JS/images (repeating the cycle above for each resource), builds the DOM, and paints the page.

Why this matters for HLD

Almost every system design concept — DNS, load balancers, CDNs, caching, connection pooling, TLS termination, stateless services — appears somewhere in this single journey. If you can explain 'what happens when I hit a URL' deeply, you already understand the skeleton of most web-scale architectures.

5. Categories of Databases and Data Infrastructure

Modern systems rarely use a single database. Each category below exists because it solves a specific access pattern well. Your job as a designer is to match the workload to the right category.

Relational Databases (SQL)

Store structured data in tables with rows and columns, enforce schemas, and support ACID transactions and joins. The default choice for core business data where consistency matters (users, orders, payments). Popular: PostgreSQL, MySQL.

Blob / Object Storage

Store large unstructured binary objects — images, videos, backups, logs, ML datasets — addressed by a key, with virtually unlimited scale and very low cost per GB. Not for querying, only storing and retrieving whole objects. Popular: Amazon S3 (also Azure Blob Storage, Google Cloud Storage).

Time Series Databases

Optimized for data points indexed by time — metrics, sensor readings, stock ticks. They excel at high-volume appends, time-range queries, downsampling, and retention policies. Popular: InfluxDB, Prometheus, TimescaleDB.

Search Databases

Build inverted indexes over text to support full-text search, relevance ranking, fuzzy matching, and faceted filtering — things relational databases do poorly at scale. Popular: Elasticsearch (also OpenSearch, Apache Solr).

Vector Databases

Store high-dimensional embedding vectors and answer similarity (nearest-neighbor) queries. Core infrastructure for AI applications: semantic search, recommendations, RAG for LLMs. Popular: Pinecone, Milvus, Weaviate, pgvector.

Wide-Column Databases

Store data in column families with flexible schemas, partitioned across many nodes for massive horizontal scale and high write throughput. Great for very large datasets with known access patterns (feeds, messaging, sensor data). Popular: Apache Cassandra (also HBase, Google Bigtable, ScyllaDB).

Caches / In-Memory Databases

Keep hot data in RAM for microsecond–millisecond reads, reducing load on primary databases. Used for session storage, rate limiting, leaderboards, and caching expensive query results. Popular: Redis (also Memcached).

Queues and Streaming Platforms

Decouple producers from consumers so systems can absorb spikes and process work asynchronously. Message queues (RabbitMQ) deliver tasks to workers; distributed logs/streams (Apache Kafka) retain ordered event streams that many consumers can replay — the backbone of event-driven architectures.

Homework Exercise 1: Go Deep on One Technology per Category

Your task

For each category above, pick the most popular technology (suggestions: PostgreSQL, S3, InfluxDB or Prometheus, Elasticsearch, Pinecone or pgvector, Cassandra, Redis, Kafka and RabbitMQ) and learn about it.

For each one, be able to answer: What problem does it solve? What is its data model? How does it scale? What are 2–3 real-world use cases? When would you NOT use it?

Bonus: install one of them locally (Redis and PostgreSQL are the easiest) and try basic operations.

6. Cost Estimation in High-Level Design

Cost is a first-class design constraint, not an afterthought. During HLD, senior engineers produce rough, back-of-the-envelope cost estimates so that reviewers can judge whether the design is economically sensible. You are not aiming for accounting precision — order-of-magnitude estimates are enough to compare alternatives.

What to estimate:

- Traffic: requests per second (average and peak), and data transferred per request.
- Storage: how much data you store today, and its growth per month/year (including replication and backups).
- Compute: how many servers/containers are needed to serve peak traffic with headroom.
- Network egress: data leaving the cloud provider is often surprisingly expensive.
- Managed service costs: databases, queues, caches, CDNs — each has its own pricing dimensions.

A useful habit: express cost per unit of business value — e.g., cost per 1,000 requests, per active user, or per GB stored. This makes designs comparable and reveals when a 'clever' architecture is actually too expensive to run.

7. Homework Exercise 2: Explore Cloud Providers

Create free-tier accounts on the major cloud providers — AWS, Microsoft Azure, and Google Cloud Platform (GCP) — and experiment. The goal is to build a high-level mental map of what services exist and how billing works, because every real HLD ultimately gets deployed on infrastructure like this.

What to do:

9. Sign up for the free tier of each provider (all three offer generous free tiers; set up billing alerts so you never get surprised).
10. Browse the service catalog and map services to the database categories from Section 5.
11. Try 2–3 services hands-on — for example, upload a file to S3, run a small compute instance, or deploy a container.
12. Study how billing works for each service you touch. Notice the different pricing models:
 - Per-request pricing — you pay per API call or invocation (e.g., AWS Lambda, S3 requests, API Gateway).
 - Fixed / provisioned pricing — you pay for capacity per hour whether you use it or not (e.g., EC2 instances, provisioned databases).
 - Usage-based pricing — you pay per GB stored, per GB transferred, or per hour of compute consumed.

A quick reference of equivalent services across providers:

Category	AWS	Azure	GCP
Object storage	S3	Blob Storage	Cloud Storage
Virtual machines	EC2	Virtual Machines	Compute Engine
Containers	ECS / EKS	AKS / Container Apps	GKE / Cloud Run
Serverless functions	Lambda	Azure Functions	Cloud Functions
Relational DB (managed)	RDS / Aurora	Azure SQL / Database for PostgreSQL	Cloud SQL
NoSQL / wide-column	DynamoDB	Cosmos DB	Bigtable / Firestore
Cache	ElastiCache	Azure Cache for Redis	Memorystore
Queues / streaming	SQS / Kinesis / MSK	Service Bus / Event Hubs	Pub/Sub
CDN	CloudFront	Azure CDN / Front Door	Cloud CDN

Deliverable for next class

Come prepared to explain: (a) one service you tried hands-on and what you built, and (b) how that service is billed and roughly what it would cost at 1 million requests/users per month.

8. Homework Summary

#	Exercise	What to produce
1	How the internet works	Be able to explain end to end what happens when you hit a URL in a browser.
2	Database categories deep-dive	Pick the most popular technology in each category (Section 5) and learn its data model, scaling, use cases, and limitations.
3	Cloud provider exploration	Create free-tier accounts on AWS, Azure, and GCP; try a few services; understand their billing models (per-request vs fixed vs usage-based).